CS 360
Programming Languages
Day 17

Review

- DSLs or “little languages”
- Language-oriented Programming
- S-expressions and Modules
- Syntax and Semantics!
- Readers and Expanders

The Reader

- Remember: the reader should consume some text and read it as Racket code, somehow
- See **notes/stacker/first_reading.rkt**
- See **notes/stacker/second_reading.rkt**
- How will the reader do this?
- See **notes/stacker/first_reader.rkt** and **notes/stacker/example_program2.rkt**
- Note that the reader produces a module!

The Reader: To-Do

- Define the **src-datums**: what should source lines look like as Racket code?
- Define the **module-datum**: what should the source file look like as a Racket module?
- To Learn:
  - “datum” symbol, ‘: “the following is a datum”
  - “insert” symbol, ~: “replace the following when formatting”
  - “quasiquote” symbol, ` : just like ‘ except a comma denotes actual code
  - “unquote” symbol, , : “the following should be treated as code”
  - “unquote-splicing” symbol, ,@: “splice in a list into this datum”
- See **notes/stacker/new_symbols.rkt**

Macros

- Macros are a type of function which transform themselves into different syntax
- For example:

```
(and cond-a cond-b cond-c) ; and is a macro
```

```
(if cond-a
    (if cond-b
        cond-c
        #f)
    #f)
```

Compile Time vs Run Time

- Racket code is *compiled* automatically (you don't have to compile it yourself)
- Compiling is the process of turning human-readable code, like Racket, into machine readable code, like bytecode or machinecode
- When you *run* Racket code, you're running the *interpreter*, which executes commands in real time
- Macros get converted to different code at *compile time*

The Reader’s Output / Expander’s Input (a syntax object)

```
(module stacker-mod "stacker.rkt"
  (handle)
  (handle 4)
  (handle 8)
  (handle +)
  (handle 3)
  (handle *))
```

The Process

- Racket calls a module's (or a language's) **read-syntax** function
- This produces a syntax object (see previous slide)
- Then, Racket calls a **macro** called **#%module-begin**
- This function is either in the reader's module or referenced by the reader's module

#%module-begin

- ALL Racket files include (implicitly) a #%module-begin, which wraps all top-level expressions
- The code inside the module produced by the reader gets passed to **#%module-begin**

```
(#%module-begin
  (handle)
  (handle 4)
  (handle 8)
  (handle +)
  (handle 3)
  (handle *))
```

- Then, like macros do, this macro rewrites the code!

The Expander

- The **expander** is a macro!
- See **notes/stacker/stacker.rkt** for the full reader/expander code
- The expander turns the expressions from the reader into a module
- That module consists of calls to a function **handle**, which will do everything for our language
- **handle** will make use of a **stack** data structure to store our numbers on

handle

- Now let’s program the **handle** function. There are four cases:
  - **(handle)**
  - **(handle a)** where **a** is a number
  - **(handle +)**
  - **(handle *)**
- See **notes/stacker/handle.rkt**

The Stack

- Recall: a stack is a last-in-first-out data structure.
- Fits naturally with Racket's **car** and **cdr** functions
- Let's build the stack together
- See **notes/stacker/stack.rkt**

Putting It All Together

1. `#lang stacker` tells Racket to look in **stacker.rkt** for the reader
2. The reader reads “raw” source into Racket S-expressions inside of a module:

```
(module stacker-mod “stacker.rkt”
    [S-expressions])
```

3. The expander expands the module body:
   1. It wraps that with #%module-begin
   2. It appends `(display (first stack))` to the body of the module (the S-expressions)